

CO1 AT2

ARTIFICIAL INTELLIGENCE ASSESSMENT

NAME:Anushka Sree Raj

REGNO:192571024

DATE:09-10-2026

ANSWERS:

(a) PEAS stands for Performance Measure, Environment, Actuators, and Sensors.

1. Performance Measure

- Generate a valid examination timetable.
- Ensure that conflicting courses are assigned different slots.
- Schedule all five courses.
- Minimize scheduling conflicts and satisfy all constraints.

2. Environment

- A university examination scheduling system.
- Five courses: A, B, C, D, E.
- Three available exam slots: Slot1, Slot2, Slot3.
- Student enrolment conflicts between specified pairs of courses.

3. Actuators

- Assign a course to an exam slot.
- Change or remove an assignment during the search process.
- Publish the final examination timetable.

4. Sensors

- Course list and available exam slots.
- Student enrolment and conflict information.
- Current partial timetable.
- Constraint-checking results.

Conclusion: The PEAS description defines the scheduling goal, the environment, the actions available to the agent, and the information it uses to generate a conflict-free timetable.

PEAS element	Description
Performance Measure	Produce a valid timetable; schedule all courses; ensure every conflicting pair has different slots. The primary requirement is zero conflicts.
Environment	University exam scheduling system; courses A, B, C, D, E; three exam slots; student-enrolment conflicts between the specified pairs.
Actuators	Assign a course to a slot; revise or remove an assignment during search; publish or export the final timetable.
Sensors	Course list, available slots, student-enrolment/conflict information, current partial timetable, and constraint-checking results.

(b) preferred intelligent agent for scheduling tasks:

The most suitable agent for this problem is a Goal-Based Agent.

A goal-based agent selects actions by considering the desired goal and finding a sequence of actions that achieves it.

Justification:

1. The goal is clearly defined: generate a conflict-free examination timetable.
2. The agent must assign a slot to each course while satisfying all constraints.
3. It evaluates partial assignments to determine whether they are valid.
4. It can use Backtracking Search to undo an assignment when a conflict occurs and try another slot.
5. It continues searching until a valid timetable is obtained.

Why not the other agents?

- Simple Reflex Agent: Acts based on the current situation and condition-action rules. It does not necessarily plan the complete schedule.
- Model-Based Agent: Maintains an internal representation of the current state and conflicts, but a model alone does not specify the scheduling goal.
- Utility-Based Agent: Selects solutions based on utility, such as room utilization or student convenience. These additional optimization objectives are not specified in this problem.
- Goal-Based Agent: Searches for an assignment that satisfies the specified goal and constraints, making it the best fit.

Conclusion: A goal-based agent is appropriate because it systematically searches for a timetable that satisfies all the given constraints.

(c) CSP Formulation and Constraint Graph (15 Marks)

A Constraint Satisfaction Problem (CSP) consists of variables, domains, and constraints.

i. Variables

Each variable represents one course whose examination slot must be assigned.

$$X = \{A, B, C, D, E\}$$

ii. Domains

Each course can be assigned to any of the three available slots.

$$D(A) = D(B) = D(C) = D(D) = D(E)$$

$$= \{\text{Slot1}, \text{Slot2}, \text{Slot3}\}$$

iii. Constraints

The following conflicting pairs must be assigned different slots:

$$A \neq B$$

$$A \neq C$$

$$B \neq C$$

$$B \neq D$$

$$C \neq D$$

$$C \neq E$$

$$D \neq E$$

These are binary constraints because each constraint involves two variables.

iv. Constraint Graph

A constraint graph represents the CSP using vertices and edges.

Vertices:

$$V = \{A, B, C, D, E\}$$

Each vertex represents a course.

Edges:

$$E = \{(A, B), (A, C), (B, C), (B, D), \\ (C, D), (C, E), (D, E)\}$$

Each edge indicates that the connected courses cannot share an exam slot.

Adjacency list:

Course	Adjacent courses (conflicts)
A	B, C
B	A, C, D

C	A, B, D, E
D	B, C, E
E	C, D

The problem is a CSP because each course must be assigned a value from its domain while satisfying all the specified binary constraints.

(d) Plain Backtracking Search :

Backtracking Search assigns values to variables one at a time. If an assignment violates a constraint, the algorithm rejects that value and tries the next available value.

Given order:

- Variable order: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$
- Value order: $\text{Slot1} \rightarrow \text{Slot2} \rightarrow \text{Slot3}$

For simplicity, let $S1 = \text{Slot1}$, $S2 = \text{Slot2}$, and $S3 = \text{Slot3}$.

Step-by-step assignment attempts

Step	Assignment attempt	Result and reason	Current partial assignment
1	A = S1	Accepted; no previous assignments.	A = S1
2	B = S1	Failed; A and B conflict.	A = S1; B unassigned
3	B = S2	Accepted; B differs from A.	A = S1, B = S2
4	C = S1	Failed; C conflicts with A.	A = S1, B = S2; C unassigned
5	C = S2	Failed; C conflicts with B.	A = S1, B = S2; C unassigned
6	C = S3	Accepted; C differs from A and B.	A = S1, B = S2, C = S3
7	D = S1	Accepted; D differs from B and C.	A = S1, B = S2, C = S3, D = S1
8	E = S1	Failed; E conflicts with D.	E unassigned
9	E = S2	Accepted; E differs from C and D.	A = S1, B = S2, C = S3, D = S1, E = S2

Explanation

- Assign A to Slot1.
- Try B in Slot1. This fails because A and B conflict. Assign B to Slot2.
- Try C in Slot1. This fails because C conflicts with A.
- Try C in Slot2. This fails because C conflicts with B.
- Assign C to Slot3, which satisfies both constraints.
- Assign D to Slot1. D differs from both B and C.

vii. Try E in Slot1. This fails because E conflicts with D.

viii. Assign E to Slot2. E differs from C and D, so all constraints are satisfied.

No variable exhausts its domain, so no backtracking to an earlier variable is required in this run.

Final timetable using Backtracking Search

Course	Course name	Assigned slot
A	Artificial Intelligence	Slot1
B	Database Systems	Slot2
C	Computer Networks	Slot3
D	Operating Systems	Slot1
E	Software Engineering	Slot2

Validation of constraints:

- A–B: Slot1 $\neq$ Slot2
- A–C: Slot1 $\neq$ Slot3
- B–C: Slot2 $\neq$ Slot3
- B–D: Slot2 $\neq$ Slot1
- C–D: Slot3 $\neq$ Slot1
- C–E: Slot3 $\neq$ Slot2
- D–E: Slot1 $\neq$ Slot2

All constraints are satisfied.

A valid timetable is obtained after 9 candidate-value attempts, of which 4 are rejected. The number of variable-level backtracks is 0.

(e) Backtracking Search with Forward Checking (20 Marks)

Forward Checking improves Backtracking Search by removing values from the domains of unassigned neighbouring variables immediately after an assignment.

If any unassigned variable has an empty domain, the algorithm must backtrack.

Initial domains:

$$D(A) = D(B) = D(C) = D(D) = D(E)$$

$$= \{S, 1, S2S3\}$$

The same variable order $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$ and value order $S1 \rightarrow S2 \rightarrow S3$ are used.

Step-by-step domain pruning

Step	Assignment	Pruning after assignment	Remaining unassigned domains
0	Start	No pruning.	A,B,C,D,E each {S1,S2,S3}
1	A=S1	Remove S1 from B and C because A conflicts with both.	B={S2,S3}; C={S2,S3}; D={S1,S2,S3}; E={S1,S2,S3}
2	B=S2	Remove S2 from C and D.	C={S3}; D={S1,S3}; E={S1,S2,S3}
3	C=S3	Remove S3 from D and E.	D={S1}; E={S1,S2}
4	D=S1	Remove S1 from E.	E={S2}
5	E=S2	All variables assigned; all constraints satisfied.	None

Explanation

Step 1: Assign A = S1

Since A conflicts with B and C, Slot1 is removed from the domains of B and C.

- B =
- C =

Step 2: Assign B = S2

B conflicts with C and D. Slot2 is removed from both domains.

- C =
- D =

Step 3: Assign C = S3

C conflicts with D and E. Slot3 is removed from both domains.

- D =
- E =

Step 4: Assign D = S1

D conflicts with E, so Slot1 is removed from E.

- E =

Step 5: Assign E = S2

All courses have been assigned slots, and every constraint is satisfied.

Final timetable using Forward Checking

Course	Course name	Assigned slot
A	Artificial Intelligence	Slot1
B	Database Systems	Slot2

C	Computer Networks	Slot3
D	Operating Systems	Slot1
E	Software Engineering	Slot2

Forward Checking obtains a valid timetable with 5 successful assignment attempts, no failed assignment attempts, and no variable-level backtracks.

(f)

Measure	Plain backtracking	Backtracking + forward checking
Successful assignments	5: A=S1, B=S2, C=S3, D=S1, E=S2.	5: A=S1, B=S2, C=S3, D=S1, E=S2.
Candidate attempts	9 values actually tried.	5 values assigned; conflicting values are pruned before trial.
Rejected candidate attempts	4: B=S1, C=S1, C=S2, E=S1.	0 attempted values fail in this run.
Variable-level backtracks	0; each variable receives a consistent value before its domain is exhausted.	0; no unassigned domain becomes empty.
Constraint processing	Checks a candidate against already assigned neighbours.	Checks conflicts and propagates their effects to unassigned neighbours' domains.
Efficiency	More candidate values are explicitly tested.	Less search effort for this instance, with extra overhead to maintain domains.

Counting convention: A candidate-value attempt means a value actually tried for a variable. A variable-level backtrack means returning to an earlier variable because the current variable has no valid value remaining.

Under these definitions, plain Backtracking Search makes 9 attempts and rejects 4, while Forward Checking makes 5 assignment attempts and rejects 0. Both use 0 variable-level backtracks for the given variable and value order.

Forward Checking reduce search effort because:

- Early constraint detection: It removes values that conflict with newly assigned courses before those values are attempted.
- Reduced search: It avoids trying assignments already known to be invalid.
- Domain wipeout detection: If a variable has no remaining values, the algorithm can detect the failure early and backtrack.
- Improved efficiency: It reduces the number of explicit assignment attempts required to find a solution.

In this example, Forward Checking removes four conflicting candidate values from consideration before they need to be tried.

Both algorithms produce the same valid examination timetable. However, Backtracking Search with Forward Checking is more efficient for this variable and value order because it prunes invalid values immediately and reduces the number of candidate assignments tested.

For larger Constraint Satisfaction Problems, Forward Checking can substantially reduce unnecessary exploration of the search space.